

Week 1

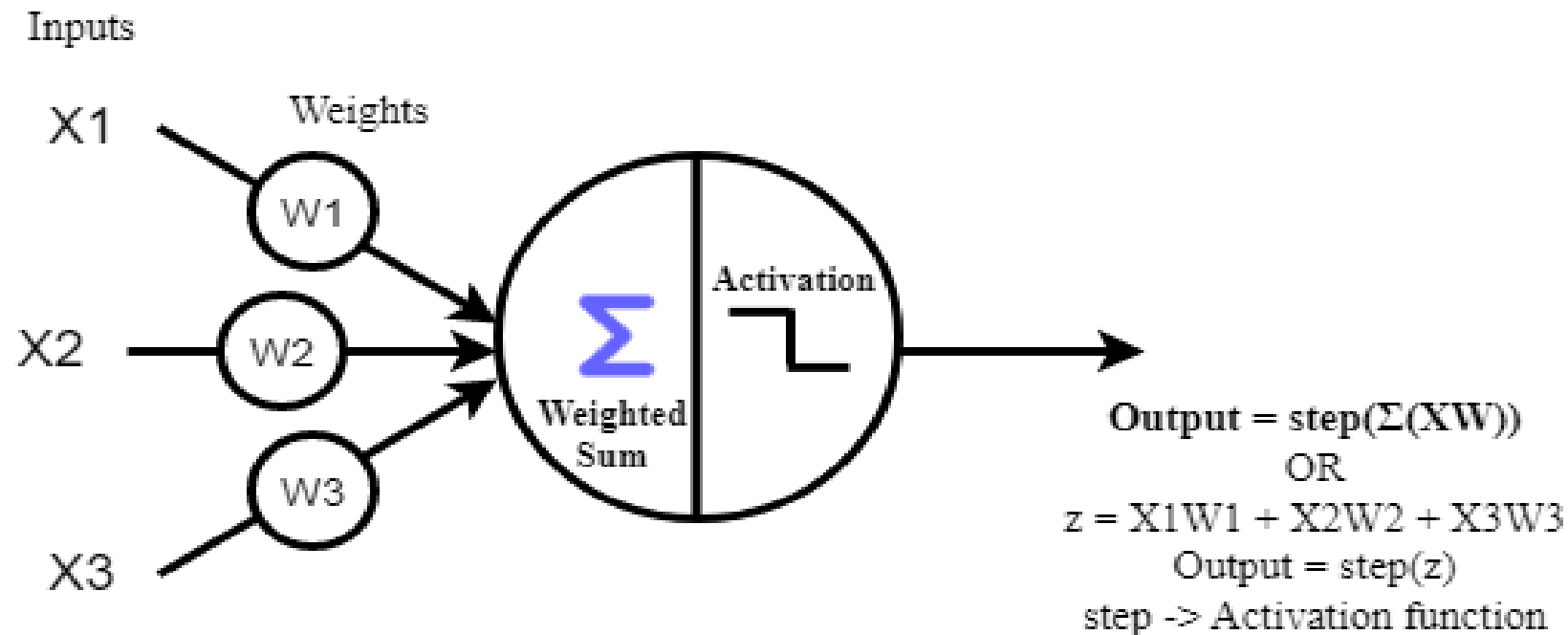
Perceptron Learning Implementation

What is Perceptron and how it has been developed?

- ❖ The word Perceptron is first ever coined in **1943** by **Warren McCulloch and Walter Pitts**. **Lately, Frank Rosenblatt**, an American psychologist, and computer scientist built the first Perceptron machine when doing research at **Cornell Aeronautical Laboratory** for performing image recognition.
- ❖ During this time the Perceptron machine become popular among the AI community and is considered the fundamental part of building intelligent systems.
- ❖ **The perceptron algorithm is based on the concept of a single neuron in the human brain, a single neuron in the human brain is doing a very simple thing, just receiving some inputs and if the inputs are high, it activates and send the signal to the next neuron.**
- ❖ The perceptron was designed to **mimic (copycat) this process**, with the input data serving as the input to the neuron and the weights representing the strength of the connections between the input neurons and the output neuron.

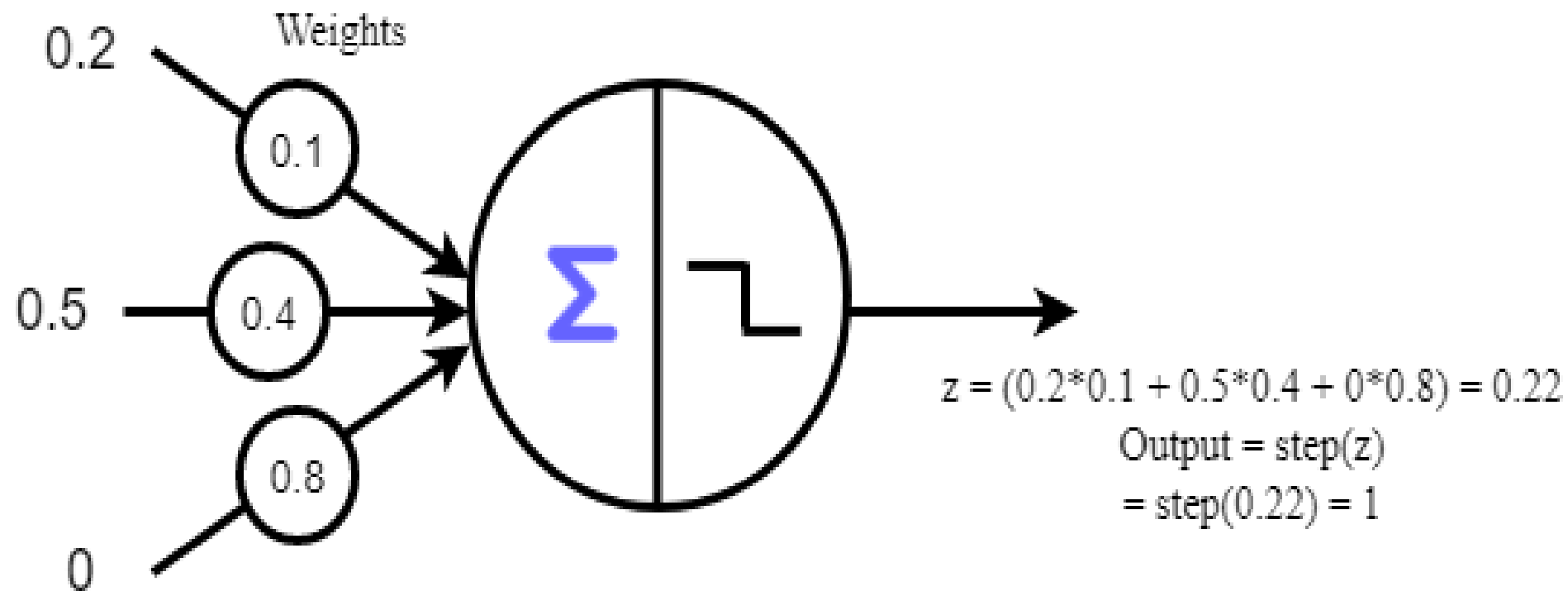
The Perceptron Algorithm, How does it work?

- ❖ The Perceptron is a type of linear classifier (binary classifier), which means it can be used to classify data that is linearly separable.
- ❖ A Perceptron is somehow similar to Logistic Regression at first glance, but it's different. While Logistic Regression predicts probabilities of a data point falling in a particular class, Perceptron will only tell whether the data point is in a particular class or not, Just like saying "Yes" or "No". Here is the diagrammatic representation of the perceptron algorithm.



- ❖ A Perceptron is a kind of single Artificial Neuron which is also known as a Threshold Processing Unit / Threshold Logic Unit (TLU).
- ❖ As you can see in the above diagram, the Perceptron contains some input links X_1 , X_2 , and, X_3 .
- ❖ Each input has its own corresponding weights W_1 , W_2 , and W_3 . These weights are basically the hearts of Perceptrons which determine the strength of each input signal to it.
- ❖ The Perceptron or Threshold Logic Unit (TLU) computes the weighted sum of the inputs ($z = X_1W_1 + X_2W_2 + X_3W_3 \dots + X_nW_n$) and then these weighted sums of inputs are passed through an activation function also known as the step function.
- ❖ These activation functions will determine whether the Perceptron needs to be activated or not.
- ❖ Let's see an example to understand this,

Inputs



Heaviside function

- ❖ The Heaviside activation function will return 0 when the weighted sum of inputs is less than zero and return 1 if it is greater than or equal to 0.]

$$\text{heaviside}(z) = \left\{ \begin{array}{ll} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{array} \right\}$$

Sign function

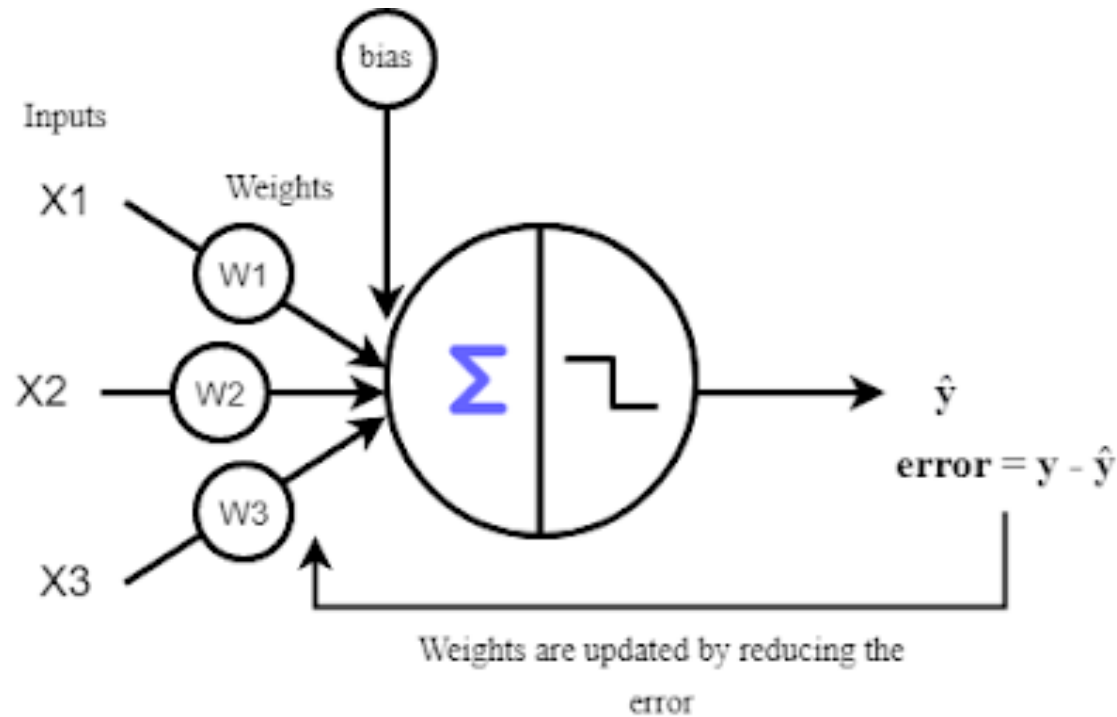
- ❖ The Sign function will return 0 if the weighted sum of inputs is 0 and return +1 and -1 when the weighted sum of inputs is greater and lesser than 0 respectively.

$$\text{sgn}(z) = \left\{ \begin{array}{lll} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{array} \right\}$$

Learning in Perceptron

- ❖ The whole point of training the perceptron is to adjust the weights of each input according to the training data.
- ❖ For training the perceptron to find the best weights, Frank Rosenblatt proposed an algorithm that is largely inspired by Hebb's rule.
- ❖ **Hebb's rule state that when a biological neuron triggers another neuron, the connection between these two neurons grows stronger.**
- ❖ There is an awesome phrase for this property of neurons, ie, **Neurons that fire together wire together.**
- ❖ Inspired by Hebb's rule, the Perceptron can be trained by finding the error made by the algorithm and finding the weights that reduce this error as much as possible.
- ❖ This is the main goal of training a machine learning algorithm in general.

- ❖ The algorithm will first make a prediction based on the training data given, then this prediction is compared with the actual desired output.
- ❖ The error is calculated for each of the training instances by comparing the prediction and actual output, finally, this error is reduced during the training process and the weights corresponding to the correct predictions are adjusted accordingly.



y = Actual values
 $\hat{y}$ = Prediction

❖ The learning(training) rule can be written as

$$w_{i,j}(next) = w_{i,j} + \eta(y_i - \hat{y})x_i$$

- ❖ Where $w_{i,j}$ is the weights between i th input neuron and j th output neuron
 - ❖ **η (eta) is the learning rate**
 - ❖ y is the target output or the actual values
 - ❖ $\hat{y}$ is the predicted value of the output neuron
 - ❖ x_i is the input value of the current training instance
- ❖ The algorithm is being trained by updating the weights until getting the optimal results.
- ❖ The learning rate(η) is required to determine the length of each step taken for finding optimal weights. Sometimes, it is better to have a smaller learning rate to get a minimum error, but sometimes a large learning rate is also preferred.

Single Layer Perceptron Model:

- ❖ This is one of the easiest Artificial neural networks (ANN) types.
- ❖ A single-layered perceptron model consists feed-forward network and also includes a threshold transfer function inside the model.
- ❖ The main objective of the single-layer perceptron model is to analyze the linearly separable objects with binary outcomes.
- ❖ In a single layer perceptron model, its algorithms do not contain recorded data, so it begins with inconstantly allocated input for weight parameters.
- ❖ Further, it sums up all inputs (weight). After adding all inputs, if the total sum of all inputs is more than a pre-determined value, the model gets activated and shows the output value as +1.

Implementing Perceptron in Python

epoch

- ❖ In deep learning, an epoch is a single pass of all training data through an algorithm.
- ❖ It's a hyperparameter that determines the training process for a machine learning model.
- ❖ During an epoch, the training data is broken down into smaller batches, called batches, and fed into the model one at a time.
- ❖ A forward pass and a backward pass together count as one pass.
- ❖ The model uses backpropagation to update its parameters throughout each epoch to lower the loss function, which measures how well the model performs on the training data.
- ❖ The procedure is repeated until the model's error rate is reduced to an acceptable level, which can involve several thousand epochs.
- ❖ The number of epochs specifies how long the network will be trained, but not in terms of time.
- ❖ **The number of epochs is a hyperparameter that defines the number times that the learning algorithm will work through the entire training dataset**

- ❖ Early stopping is a technique that can help find a good balance for the number of epochs to run.
- ❖ It involves monitoring the model's performance on a validation set and stopping the training process when performance starts to decline.
- ❖ This approach helps ensure that the model is neither underfit nor overfit.
- ❖ An epoch is when all the training data is used at once and is defined as the total number of iterations of all the training data in one cycle for training the machine learning model.
- ❖ **Another way to define an epoch is the number of passes a training dataset takes around an algorithm.**
- ❖ **An epoch means training the neural network with all the training data for one cycle. In an epoch, we use all of the data exactly once. A forward pass and a backward pass together are counted as one pass: An epoch is made up of one or more batches, where we use a part of the dataset to train the neural network.**

1. `import numpy as np`
2. `import matplotlib.pyplot as plt`
3. `import tensorflow as tf`
4. `from sklearn.datasets import load_iris`
5. `from sklearn.model_selection import train_test_split`
6. `from sklearn.preprocessing import StandardScaler`
7. `from sklearn.metrics import classification_report`

❖ We import the necessary libraries: `numpy` for numerical operations, `matplotlib` for plotting, **`tensorflow` for creating and training the model**, and various `scikit-learn` modules for dataset loading, preprocessing, and metrics.

TensorFlow is an open-source framework for building and training machine learning models, especially neural networks. It uses tensors (multi-dimensional arrays) to perform computations efficiently, and it's popular for tasks like image recognition, language processing, and more. It offers flexibility, distributed computing, and high-level APIs like Keras.

1. `# Load Iris dataset`
2. `iris = load_iris()`
3. `X = iris.data`
4. `y = (iris.target == 0).astype(int) # Setosa vs non-Setosa`

- ❖ We load the Iris dataset using scikit-learn's `load_iris` function. `X` contains the features, and `y` contains the target labels.
- ❖ We're specifically interested in classifying Setosa versus non-Setosa flowers, so we convert the original target labels into binary labels using `(iris.target == 0).astype(int)`.

1. # Split the dataset into training and testing sets
 2. `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)`
- ❖ We split the dataset into training and testing sets using `train_test_split`.
 - ❖ The testing set will be 30% of the total data, and we set a random seed of 42 for reproducibility.
1. # Standardize the features
 2. `scaler = StandardScaler()`
 3. `X_train = scaler.fit_transform(X_train)`
 4. `X_test = scaler.transform(X_test)`
- ❖ We standardize the features using `StandardScaler` from `scikit-learn`. Standardization ensures that the features have a mean of 0 and a standard deviation of 1.
 - ❖ This step helps the model converge faster and perform better.

1. `# Build the single-layer Perceptron model using TensorFlow`
2. `model = tf.keras.Sequential`
3. `([`
4. `tf.keras.layers.Dense(1, activation='sigmoid', input_shape=(X_train.shape[1],))`
5. `])`

- ❖ We create a single-layer Perceptron model using TensorFlow's Keras API. The model consists of a single neuron (`Dense(1)`) with a sigmoid activation function.
- ❖ The `input_shape` is set to the number of features in the dataset (`X_train.shape[1]`).

1. `# Compile the model`
2. `model.compile(optimizer='sgd', loss='binary_crossentropy', metrics=['accuracy'])`

- ❖ We compile the model with the stochastic gradient descent (SGD) optimizer and the binary cross-entropy loss function.
- ❖ We also specify that we want to track the accuracy as a metric during training.

1. `# Train the model`
2. `history = model.fit(X_train, y_train, epochs=100, verbose=1)`

- ❖ We train the model on the training data. The fit function is used for training.
- ❖ We specify the number of epochs (100) and set verbose to 1 to see the training progress.

1. `# Predict on the test set`
2. `predictions = model.predict(X_test)`
3. `binary_predictions = (predictions > 0.5).astype(int)`

- ❖ We predict the labels on the test set using the trained model.
- ❖ Since the sigmoid activation function was used, we set a threshold of 0.5 to convert the continuous predictions into binary predictions.

1. `# Generate classification report`
2. `class_names = ['Non-Setosa', 'Setosa'] # Order corresponds to the labels`
3. `report = classification_report(y_test, binary_predictions, target_names=class_names)`

- ❖ We generate a classification report using scikit-learn's `classification_report` function. The `target_names` parameter is used to specify the names of the classes.

1. `print("Classification Report:\n")`
2. `print(report)`

❖ We print the classification report to the console.

1. `# Plot the decision boundary`
2. `plt.figure(figsize=(8, 6))`
3. `plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap='coolwarm', marker='o', label='True Class')`
4. `plt.xlabel('Sepal Length')`
5. `plt.ylabel('Sepal Width')`
6. `x_vals = np.linspace(X_test[:, 0].min(), X_test[:, 0].max(), 100)`
7. `y_vals = (-model.weights[0][0] * x_vals - model.weights[1]) / model.weights[0][1]`
8. `plt.plot(x_vals, y_vals, 'r--', label='Decision Boundary')`
9. `plt.legend()`
10. `plt.show()`

- ❖ We plot the decision boundary using matplotlib.
- ❖ First, we scatter plot the test samples with their true classes using different colors.
- ❖ Then, we calculate the decision boundary line using the Perceptron's weights and plot it as a red dashed line.
- ❖ The plot shows the Sepal Length vs. Sepal Width features along with the decision boundary.